

Code Similarity Platform

Maintenance Manual

Version: 1.0

Prepared by: Group 16 — Muhammad Nabeel Waheed, Thomas Neal,
Ghassan Balouze, Kev Akpinar, Abdel Zahran, Ty Mabee

Kool Group Name: Last Minute Warriors

Course: COSC 4P02 — Course Project

2026-04-24

Contents

1	System Requirements	3
1.1	Hardware Requirements	3
1.2	Operating System Requirements	3
1.3	Runtime Dependency Versions	3
1.4	Network and Port Requirements	4
2	Installation & Initial Setup	5
2.1	Docker Compose Installation (Recommended)	5
2.1.1	Prerequisites	5
2.1.2	Step-by-Step Installation	5
2.1.3	Environment Configuration	6
2.1.4	Database Initialization	7
2.1.5	First-Run Verification Checklist	7
3	Routine Maintenance Procedures	8
3.1	Database Backup and Restore	8
3.1.1	Backup	8
3.1.2	Restore	8
3.2	Log Management	9
3.3	Dependency Update Process	10
3.3.1	Node.js Dependencies	10
3.3.2	Rust Dependencies	10
3.4	Database Migration Procedure	11
3.5	Certificate and Secret Rotation	11
3.5.1	TLS Certificates (Caddy)	11
3.5.2	Session Cookie Secret	11
3.5.3	RSA Submission Identity Key Rotation	11
3.5.4	Public Upload Status Token Secret	12
3.6	Performance Monitoring	12
3.7	Scheduled Tasks and Cron Jobs	13
4	System Health Monitoring	15
4.1	Key Health Indicators per Component	15
4.2	How to Check System Status	15
4.3	Log File Locations and Interpretation	16
4.4	Expected Baseline Performance	17
5	Scaling & Capacity	18
5.1	Horizontal Scaling	18
5.2	Known Bottlenecks	19
5.3	Capacity Planning Considerations	19
6	Dependency Management	20

6.1	Updating Node.js Runtime	20
6.2	Updating Rust Toolchain	20
6.3	Updating PostgreSQL	20
6.4	Updating Redis	21
6.5	Tree-sitter Grammar Updates	21
7	Data Management	22
7.1	Data Retention Policies	22
7.2	Safely Purging Old Data	22
7.3	Data Export Procedures	23
8	Security Maintenance	24
8.1	Access Control Review	24
8.2	Credential Rotation Schedule	24
8.3	Audit Log Review	24
8.4	Security Hardening Checklist	25

Chapter 1. System Requirements

1.1 Hardware Requirements

The following are baseline recommendations for a single-node deployment serving a university-scale course (hundreds of students per assignment).

Component	Minimum	Recommended
API server (CPU)	1 vCPU	2 vCPU
API server (RAM)	512 MB	1 GB
Worker (CPU)	1 vCPU	2 vCPU (engine is CPU-intensive)
Worker (RAM)	512 MB	1 GB
PostgreSQL (CPU)	1 vCPU	2 vCPU
PostgreSQL (RAM)	512 MB	2 GB
Redis (RAM)	128 MB	512 MB
Web frontend (RAM)	256 MB	512 MB
Object storage (disk)	5 GB	20 GB+ (grows with uploads)

NOTE: The engine binary (`engine-cli`) is CPU-bound. A comparison run over 100 submissions generates approximately 4,950 pairs. Each pair invokes the full GST algorithm. Run time scales approximately quadratically with the number of submissions. With a single comparison concurrency worker and 100 submissions, a run may take several minutes on a single CPU core. Plan capacity accordingly.

1.2 Operating System Requirements

- **Docker deployments:** Any Linux distribution with Docker Engine 24+ and Docker Compose 2.20+. The official container images are based on `node:22-bookworm-slim` (Debian Bookworm) and `rust:1.86-bookworm`.
- **Bare-metal deployments:** Debian 12 / Ubuntu 22.04 LTS or newer. Requires Node.js 22, Rust 1.86+, PostgreSQL 16, Redis 7.

1.3 Runtime Dependency Versions

Dependency	Required Version	Notes
Node.js	22.x LTS	API, worker, and web frontend
npm	10.x	Package manager
Rust toolchain	1.86+	Building engine-cli
PostgreSQL	16	Database

Redis	7	Job queue
Caddy	2.x	Reverse proxy (production)
Docker Engine	24+	Container runtime
Docker Compose	2.20+	Container orchestration

1.4 Network and Port Requirements

Service	Port	Protocol	Accessible from
Caddy (HTTP redirect)	80	TCP	Internet
Caddy (HTTPS)	443	TCP	Internet
API (internal)	3001	TCP	Caddy, worker
Next.js web (internal)	3000	TCP	Caddy
PostgreSQL	5432	TCP	API, worker
Redis	6379	TCP	API, worker

PostgreSQL and Redis must **not** be exposed to the public internet. In the Docker Compose setup, they are on an internal network and not port-mapped to the host. In bare-metal deployments, configure firewall rules accordingly.

Chapter 2. Installation & Initial Setup

2.1 Docker Compose Installation (Recommended)

2.1.1 Prerequisites

Listing 2.1: Install Docker on Debian/Ubuntu

```
1 # Install Docker Engine
2 curl -fsSL https://get.docker.com | sh
3 sudo usermod -aG docker $USER
4 # Log out and back in
5
6 # Verify
7 docker --version
8 docker compose version
```

2.1.2 Step-by-Step Installation

Listing 2.2: Full Docker Compose installation

```
1 # Step 1: Clone repository
2 git clone <repository-url>
3 cd Anti-Vibe-Coder
4
5 # Step 2: Create environment file
6 cp .env.example .env
7
8 # Step 3: Edit .env -- set all required variables (see Section
9 2.3)
10 nano .env
11
12 # Step 4: Generate RSA key pair for submission identity
13 # (if not already done -- this only needs to be done once)
14 node -e "
15 const crypto = require('crypto');
16 const { publicKey, privateKey } = crypto.generateKeyPairSync('rsa', {
17   modulusLength: 4096,
18   publicKeyEncoding: { type: 'spki', format: 'pem' },
19   privateKeyEncoding: { type: 'pkcs8', format: 'pem' }
20 });
21 const pub = Buffer.from(publicKey).toString('base64');
22 const priv = Buffer.from(privateKey).toString('base64');
23 console.log('SUBMISSION_IDENTITY_PUBLIC_KEY_PEM_BASE64=' + pub);
24 console.log('SUBMISSION_IDENTITY_PRIVATE_KEY_PEM_BASE64=' + priv);
25 ;
```

```

24 "
25 # Paste output into .env
26
27 # Step 5: Generate a random token secret
28 node -e "console.log(require('crypto').randomBytes(32).toString('
    hex'))"
29 # Paste as PUBLIC_UPLOAD_STATUS_TOKEN_SECRET in .env
30
31 # Step 6: Set a key ID (any non-empty alphanumeric string)
32 # SUBMISSION_IDENTITY_KEY_ID=prod-key-001
33
34 # Step 7: First-time: build and start only the database and redis
35 docker compose up -d postgres redis
36
37 # Wait for postgres to be healthy (about 10 seconds)
38 docker compose ps
39
40 # Step 8: Run database migrations (from host)
41 DATABASE_URL=$(grep DATABASE_URL .env | cut -d= -f2-) \
42 npx prisma migrate deploy --schema prisma/schema.prisma
43
44 # OR use the npm script after installing node deps:
45 npm install
46 npm run db:migrate:deploy
47
48 # Step 9: Bootstrap the first professor (admin) account
49 ADMIN_BOOTSTRAP_EMAIL=admin@example.com \
50 ADMIN_BOOTSTRAP_PASSWORD=ChangeMe123! \
51 npm run db:bootstrap-admin
52
53 # Step 10: Build and start all services
54 docker compose up --build -d
55
56 # Step 11: Check service health
57 curl http://localhost/api/health/live
58 curl http://localhost/api/health/ready

```

2.1.3 Environment Configuration

Edit `.env` and set the following. See the Technical Specification (Chapter 6) for a full reference of every variable.

Mandatory in production:

Listing 2.3: Mandatory production environment variables

```

1 NODE_ENV=production
2 DATABASE_URL=postgres://user:password@host:5432/similarity_app
3 REDIS_URL=redis://redis:6379
4 CORS_ALLOWED_ORIGINS=https://similarityapp.ca,https://www.
    similarityapp.ca
5 NEXT_PUBLIC_API_BASE_URL=https://similarityapp.ca/api

```

```

6 SUBMISSION_IDENTITY_KEY_ID=prod-key-001
7 SUBMISSION_IDENTITY_PUBLIC_KEY_PEM_BASE64=<base64-encoded-pem>
8 SUBMISSION_IDENTITY_PRIVATE_KEY_PEM_BASE64=<base64-encoded-pem>
9 PUBLIC_UPLOAD_STATUS_TOKEN_SECRET=<32-byte-hex>
10 ADMIN_BOOTSTRAP_EMAIL=admin@university.ca
11 ADMIN_BOOTSTRAP_PASSWORD=<secure-password>

```

Security-critical defaults to review:

```

1 AUTH_COOKIE_SECURE=true           # Must be true in production (
    HTTPS)
2 AUTH_COOKIE_SAME_SITE=lax         # Review for your domain setup
3 AUTH_COOKIE_DOMAIN=similarityapp.ca # Set to your domain
4 AUTH_SESSION_TTL_HOURS=168        # 7 days -- adjust to policy
5 UPLOAD_MAX_FILE_SIZE_MB=250      # Adjust to course needs

```

2.1.4 Database Initialization

Listing 2.4: Database initialization sequence

```

1 # Apply migrations
2 npm run db:migrate:deploy
3
4 # Bootstrap admin account (only needed once)
5 npm run db:bootstrap-admin
6
7 # Verify: connect to database and check tables
8 psql $DATABASE_URL -c "\dt"

```

2.1.5 First-Run Verification Checklist

GET /api/health/live returns {"status":"ok","service":"api"}

GET /api/health/ready returns {"status":"ok","checks":{"database":"ok","redis":"ok"}}

Login with bootstrap admin account works via the web UI

Creating an assignment succeeds

Uploading a ZIP archive creates a batch with status **READY** (after worker processes it)

A comparison run completes and pair results appear in the UI

The pair viewer displays highlighted code regions

Student public upload endpoint is reachable and returns a batch ID

Chapter 3. Routine Maintenance Procedures

3.1 Database Backup and Restore

NOTE: The repository contains no automated backup configuration. The following procedures should be implemented and scheduled manually.

3.1.1 Backup

Listing 3.1: PostgreSQL backup (Docker Compose)

```
1 # Full database dump (compressed)
2 docker compose exec postgres pg_dump \
3   -U postgres \
4   -d similarity_app \
5   -F c \
6   -f /tmp/similarity_app_$(date +%Y%m%d_%H%M%S).dump
7
8 # Copy dump to host
9 docker compose cp postgres:/tmp/similarity_app_*.dump ./backups/
10
11 # Alternative: direct pg_dump from host with DATABASE_URL
12 pg_dump \
13   --format=custom \
14   --compress=9 \
15   "$DATABASE_URL" \
16   > ./backups/similarity_app_$(date +%Y%m%d_%H%M%S).dump
```

Listing 3.2: Object storage backup

```
1 # Backup the object storage volume (mounted at /var/lib/docker/
   # volumes/...)
2 # Identify volume name
3 docker volume ls | grep object-storage
4
5 # Create tar archive of the volume
6 docker run --rm \
7   -v similarity-app_object-storage:/source:ro \
8   -v $(pwd)/backups:/backup \
9   busybox tar czf /backup/object-storage_$(date +%Y%m%d_%H%M%S).
   tar.gz /source
```

3.1.2 Restore

Listing 3.3: PostgreSQL restore

```
1 # Stop API and worker to prevent writes during restore
2 docker compose stop api worker
```

```
3
4 # Drop and recreate database
5 docker compose exec postgres psql -U postgres \
6   -c "DROP DATABASE IF EXISTS similarity_app;"
7 docker compose exec postgres psql -U postgres \
8   -c "CREATE DATABASE similarity_app;"
9
10 # Restore from dump
11 docker compose exec -T postgres pg_restore \
12   -U postgres \
13   -d similarity_app \
14   < ./backups/similarity_app_YYYYMMDD_HHMMSS.dump
15
16 # Restart services
17 docker compose start api worker
```

Listing 3.4: Object storage restore

```
1 # Restore object storage from tar
2 docker run --rm \
3   -v similarity-app-object-storage:/target \
4   -v $(pwd)/backups:/backup \
5   busybox tar xzf /backup/object-storage_YYYYMMDD_HHMMSS.tar.gz \
6   -C /target --strip-components=1
```

3.2 Log Management

The application does not configure a log aggregation system. All logs are written to stdout/stderr and captured by Docker.

Listing 3.5: View service logs

```
1 # Follow all service logs
2 docker compose logs -f
3
4 # View specific service logs
5 docker compose logs -f api
6 docker compose logs -f worker
7 docker compose logs -f web
8
9 # View last 100 lines
10 docker compose logs --tail=100 api
11
12 # Filter by time
13 docker compose logs --since="2026-04-24T00:00:00" api
```

NOTE: For production use, configure log forwarding to a centralised log system (e.g., Loki + Grafana, ELK stack, or CloudWatch). Docker’s default `json-file` logging driver rotates logs but does not forward them.

To configure Docker log rotation, add to `/etc/docker/daemon.json`:

```
1 {
2   "log-driver": "json-file",
3   "log-opts": {
4     "max-size": "100m",
5     "max-file": "5"
6   }
7 }
```

3.3 Dependency Update Process

3.3.1 Node.js Dependencies

Listing 3.6: Update Node.js dependencies

```
1 # Check for outdated packages
2 npm outdated
3
4 # Update all packages within semver constraints
5 npm update
6
7 # For major version updates (review changelog first):
8 npx npm-check-updates -i
9
10 # After updating, run full test suite
11 npm run test
12 cargo test --manifest-path engine/Cargo.toml --workspace
13
14 # Rebuild and redeploy
15 docker compose up --build -d
```

3.3.2 Rust Dependencies

Listing 3.7: Update Rust dependencies

```
1 # Check for updates
2 cd engine
3 cargo outdated # requires cargo-outdated: cargo install cargo-
4 outdated
5
6 # Update within Cargo.toml constraints
7 cargo update
8
9 # Run engine tests
10 cargo test --workspace
11
12 # If tree-sitter grammars are updated, verify language parsing
13 still works
14 # by running the engine-language tests specifically:
15 cargo test -p engine-language
```

3.4 Database Migration Procedure

Listing 3.8: Applying migrations in production

```
1 # 1. Take a database backup FIRST (see Section 3.1)
2
3 # 2. Put application in maintenance mode if needed
4 #   (currently no maintenance mode flag exists -- stop API and
5   worker)
6 docker compose stop api worker
7
8 # 3. Apply migrations
9 npm run db:migrate:deploy
10 # OR:
11 npx prisma migrate deploy --schema prisma/schema.prisma
12
13 # 4. Restart services
14 docker compose start api worker
15
16 # 5. Verify health endpoints
17 curl http://localhost/api/health/ready
```

Rollback: Prisma does not support automatic migration rollback. If a migration fails, restore the database from backup (Section 3.1) and redeploy the previous application version.

3.5 Certificate and Secret Rotation

3.5.1 TLS Certificates (Caddy)

Caddy automatically manages TLS certificates via Let's Encrypt. No manual rotation is required. Caddy stores certificates in the `caddy-data` Docker volume. Certificates are renewed automatically before expiry.

3.5.2 Session Cookie Secret

Sessions are validated by looking up the token hash in the database. There is no separate HMAC secret for session cookies. To invalidate all active sessions (e.g., after a security incident):

Listing 3.9: Invalidate all sessions

```
1 psql $DATABASE_URL -c "DELETE FROM \"Session\";"
```

3.5.3 RSA Submission Identity Key Rotation

Listing 3.10: Rotate RSA identity key

```
1 # 1. Generate new key pair
2 node -e "
3 const crypto = require('crypto');
```

```

4  const { publicKey, privateKey } = crypto.generateKeyPairSync('rsa
    ', {
5      modulusLength: 4096,
6      publicKeyEncoding: { type: 'spki', format: 'pem' },
7      privateKeyEncoding: { type: 'pkcs8', format: 'pem' }
8  });
9  console.log('PUBLIC:', Buffer.from(publicKey).toString('base64'))
    ;
10 console.log('PRIVATE:', Buffer.from(privateKey).toString('base64
    '));
11 "
12
13 # 2. Update .env with new key and a new
    SUBMISSION_IDENTITY_KEY_ID
14
15 # 3. Note: existing encrypted identities will no longer be
    decryptable
16 #     with the new key. If existing submissions must remain
    decryptable,
17 #     the old key must be retained.
18
19 # 4. Redeploy
20 docker compose up --build -d

```

3.5.4 Public Upload Status Token Secret

Listing 3.11: Rotate upload status token secret

```

1  # Generate new secret
2  node -e "console.log(require('crypto').randomBytes(32).toString('
    hex'))"
3
4  # Update PUBLIC_UPLOAD_STATUS_TOKEN_SECRET in .env
5  # Redeploy API
6  docker compose up -d --no-deps api

```

3.6 Performance Monitoring

NOTE: No metrics or monitoring infrastructure is configured in the repository. The following guidance describes what should be monitored.

Component	Key Metric	Warning Threshold
API	Request latency (p95)	> 2s for non-upload endpoints
API	HTTP 5xx error rate	> 1%
API	Session table row count	> 10,000 (indicates session GC not running)

Worker	BullMQ queue depth	> 20 jobs
	(upload-preparation)	
Worker	BullMQ queue depth	> 5 jobs
	(comparison-run)	
Worker	Failed job count	> 0
Engine	Comparison run duration	> 600s (single run)
PostgreSQL	Active connections	> 80% of max_connections
PostgreSQL	Table bloat / vacuum lag	Monitor with pg_stat_user_tables
Redis	Memory usage	> 80% of maxmemory
Disk	Object storage volume	> 80% full

Listing 3.12: Manual health checks

```

1 # API liveness and readiness
2 curl -s http://localhost/api/health/live | jq .
3 curl -s http://localhost/api/health/ready | jq .
4
5 # PostgreSQL connection count
6 psql $DATABASE_URL -c \
7     "SELECT count(*) FROM pg_stat_activity WHERE state = 'active';"
8
9 # Redis memory usage
10 docker compose exec redis redis-cli INFO memory | grep
    used_memory_human
11
12 # Object storage disk usage
13 docker run --rm \
14     -v similarity-app-object-storage:/storage:ro \
15     busybox du -sh /storage
16
17 # BullMQ queue depths (from Redis)
18 docker compose exec redis redis-cli LLEN bull:upload-preparation:
    wait
19 docker compose exec redis redis-cli LLEN bull:comparison-run:wait

```

3.7 Scheduled Tasks and Cron Jobs

NOTE: The codebase contains a `revokeExpiredSessions()` method in `SessionService`, but no scheduler invokes it automatically. Expired session rows will accumulate. Implement a periodic job or PostgreSQL cron to purge them.

Listing 3.13: Expired session purge (manual)

```

1 psql $DATABASE_URL -c \
2     "DELETE FROM \"Session\" WHERE \"expiresAt\" < NOW();"

```

Recommended: add this as a cron job on the server running the API, or schedule it in the application startup. The session table has an index on `expiresAt` which makes this

query efficient.

Recommended crontab entry (run daily at 02:00):

```
1 0 2 * * * psql "$DATABASE_URL" -c \  
2 "DELETE FROM \"Session\" WHERE \"expiresAt\" < NOW();" >> /var/  
   log/session-gc.log 2>&1
```

Chapter 4. System Health Monitoring

4.1 Key Health Indicators per Component

Component	Healthy Indicators	Unhealthy Indicators
API	Health endpoint returns 200; low 5xx rate; log lines show normal request processing	5xx errors; <code>PrismaClientInitializationError</code> ; Redis connection refused
Worker	No failed jobs in BullMQ; comparison runs complete within expected time; no ERROR logs	Repeated job failures; engine process exit codes $\neq 0$; failed archive extractions
PostgreSQL	Responds to <code>SELECT 1</code> ; no bloat warnings; normal connection count	Connection refused; high query latency; lock waits
Redis	Responds to <code>PING</code> ; low memory usage	Out of memory; connection refused
Engine	Zero non-zero exit codes from engine-cli	<code>stderr</code> output; non-zero exit codes from spawn
Object storage	Disk < 80% full; file creation succeeds	<code>ENOSPC</code> errors; permission errors on write
Caddy	TLS certificate valid; 80/443 ports accept connections	Certificate expiry warnings; backend connection refused

4.2 How to Check System Status

Listing 4.1: System status check commands

```
1 # All Docker services status
2 docker compose ps
3
4 # API health (external check via Caddy)
5 curl -s https://similarityapp.ca/api/health/ready | jq .
6
7 # PostgreSQL health
8 docker compose exec postgres pg_isready -U postgres
9
10 # Redis health
11 docker compose exec redis redis-cli PING
12
```



```

13 # Check for BullMQ failed jobs
14 docker compose exec redis redis-cli LLEN bull:upload-preparation:
    failed
15 docker compose exec redis redis-cli LLEN bull:comparison-run:
    failed
16
17 # Engine binary is accessible and executable
18 docker compose exec worker ls -la /app/bin/engine-cli
19 docker compose exec worker /app/bin/engine-cli --help 2>&1 ||
    true
20 # (engine-cli exits 1 on bad input but should at least respond)

```

4.3 Log File Locations and Interpretation

In the Docker Compose setup, all logs are captured by Docker. Access them with:

```
1 docker compose logs [service]
```

Service	Log access	What to look for
api	docker compose logs api	HTTP request lines (Fastify), NestJS module initialization, [Error] lines, Prisma query errors
worker	docker compose logs worker	BullMQ job start/complete/-fail messages, engine spawn output, archive extraction warnings
web	docker compose logs web	Next.js server errors, build output
postgres	docker compose logs postgres	Connection errors, slow query warnings
redis	docker compose logs redis	Memory warnings, persistence errors
caddy	docker compose logs caddy	TLS certificate events, reverse proxy errors

Common API log patterns and their meaning:

Listing 4.2: API log pattern examples

```

1 # Normal request (Fastify)
2 {"level":"info","time":...,"method":"GET","url":"/health/ready","
    statusCode":200}
3
4 # Session validation failure (expected if cookie expired)
5 UnauthorizedException: Session is invalid or expired
6
7 # Database connection failure (critical)

```

```

8 PrismaClientInitializationError: Unable to connect to the
   database server
9
10 # Upload too large (expected user error)
11 RequestFileTooLargeError

```

4.4 Expected Baseline Performance

Operation	Expected duration	Notes
Login / health check	< 100ms	
Assignment list	< 200ms	
Upload batch creation (metadata)	< 300ms	
Archive upload (250 MB)	Upload-speed limited	Streaming write
Upload preparation (single 5 MB archive)	2–15 seconds	Depends on file count
Comparison run (10 submissions)	10–60 seconds	
Comparison run (100 submissions)	5–30 minutes	Engine is CPU-bound
Pair result query	< 500ms	

Chapter 5. Scaling & Capacity

5.1 Horizontal Scaling

Component	Can scale horizontally?	Notes
API	Yes	Stateless. Multiple instances behind a load balancer. Session state is in PostgreSQL. The object storage volume must be a shared network filesystem (NFS or equivalent) if multiple API instances write uploads.
Worker	Yes (with caveats)	Multiple worker instances consume from the same Redis queues. Concurrency is controlled per-worker by <code>WORKER_COMPARISON_CONCURRENCY</code> . The engine binary must be present on each worker host. Object storage must be shared (see API note).
Engine	N/A	Runs as a child process of the worker. Scale by running more workers.
Web frontend	Yes	Stateless Next.js server.
PostgreSQL	Limited	Single primary. Read replicas can be added for reporting queries. Prisma does not natively support read replicas without custom configuration.
Redis	Limited	Single instance is sufficient for BullMQ. Redis Cluster is supported by BullMQ if needed.

5.2 Known Bottlenecks

- **Engine CPU usage:** The GST algorithm is $O(n \cdot m)$ per pair in the worst case. Large assignments (100+ submissions) generate thousands of pairs. The engine is single-threaded per invocation. The primary scaling lever is `WORKER_COMPARISON_CONCURRENCY` (parallel comparison runs across assignments) and running additional worker instances.
- **Object storage:** Currently filesystem-backed (local volume). A shared network filesystem is required for multi-node deployments. Cloud object storage (S3-compatible) is not yet implemented.
- **Prisma connection pool:** Prisma uses a connection pool per process. Under high concurrency, pool exhaustion can occur. Tune using `connection_limit` in the `DATABASE_URL` query string.
- **flate ZIP extraction:** The worker extracts ZIP archives synchronously in the Node.js thread. Very large archives (near the 50 MB limit) may cause temporary event-loop stalls. This is mitigated by the configurable concurrency limit.

5.3 Capacity Planning Considerations

- **Disk:** Each student submission stores the original ZIP (object storage) plus the extracted source files. Estimate roughly $2\text{--}3\times$ the archive size per submission.
- **Database:** `PairMatch` rows are the most numerous. A full comparison of 100 submissions ($\approx 4,950$ pairs, each with ~ 20 matches on average) creates $\approx 100,000$ rows per comparison run. Monitor table size using PostgreSQL's `pg_table_size()`.
- **Redis:** BullMQ stores job payloads in Redis. For large comparison runs, job payloads can be several megabytes (all concatenated sources). Monitor Redis memory with `redis-cli INFO memory`.

Chapter 6. Dependency Management

6.1 Updating Node.js Runtime

Listing 6.1: Update Node.js version in Docker images

```
1 # The Dockerfiles specify node:22-bookworm-slim
2 # To update to a newer LTS, change the FROM line in all three
   Dockerfiles:
3 # Dockerfile.api, Dockerfile.worker, Dockerfile.web
4
5 # Example: update to Node.js 24 LTS
6 sed -i 's/node:22-bookworm-slim/node:24-bookworm-slim/g'
   Dockerfile.*
7
8 # Rebuild and test
9 docker compose up --build -d
10 npm run test
```

6.2 Updating Rust Toolchain

Listing 6.2: Update Rust version

```
1 # Update the FROM line in Dockerfile.worker
2 sed -i 's/rust:1.86-bookworm/rust:1.88-bookworm/g' Dockerfile.
   worker
3
4 # Test locally
5 cargo test --manifest-path engine/Cargo.toml --workspace
6
7 # Rebuild Docker image
8 docker compose build worker
```

6.3 Updating PostgreSQL

PostgreSQL major version upgrades require a `pg_upgrade` procedure or dump-and-restore. Minor version upgrades (e.g., 16.1 to 16.4) are safe and can be done by pulling a new Docker image:

Listing 6.3: PostgreSQL minor version update

```
1 # Take backup first
2 docker compose exec postgres pg_dump -U postgres -F c
   similarity_app \
3 > backup_before_upgrade_$(date +%Y%m%d).dump
4
5 # Pull new image
```

```
6 docker compose pull postgres
7
8 # Restart (minor versions are compatible)
9 docker compose up -d postgres
```

6.4 Updating Redis

Listing 6.4: Redis update

```
1 docker compose pull redis
2 docker compose up -d redis
3 # Redis is backward-compatible across minor versions within major
  (e.g., 7.x)
```

6.5 Tree-sitter Grammar Updates

The engine uses tree-sitter grammars for Java, C, and C++ (declared in `engine/Cargo.toml`). Grammar updates may improve parsing accuracy or add support for new language constructs.

Listing 6.5: Update tree-sitter grammars

```
1 cd engine
2 cargo update -p tree-sitter-java
3 cargo update -p tree-sitter-c
4 cargo update -p tree-sitter-cpp
5
6 # Run all engine tests to verify no regressions
7 cargo test --workspace
8
9 # Rebuild Docker image
10 docker compose build worker
```

Chapter 7. Data Management

7.1 Data Retention Policies

NOTE: No data retention policies are implemented in the codebase. The following recommendations should be implemented based on institutional requirements.

Recommended retention considerations:

- **Submissions and comparison results:** Retain for the duration of the course plus any academic integrity investigation periods (typically 1–2 academic years).
- **Session records:** Purge expired sessions daily (see Section 3.7).
- **Uploaded archives (object storage):** Consider purging original ZIP archives after successful preparation (submissions are already parsed and stored). The `originalObjectKey` on `UploadBatch` references the staged archive.
- **Failed upload batches:** Failed batches do not create submission records. The original object key may reference a file that should be cleaned up.

7.2 Safely Purging Old Data

To delete an assignment and all associated data:

Listing 7.1: Delete assignment via API (professor)

```
1 # Via the API (recommended -- handles object key cleanup)
2 curl -X DELETE \
3     -H "Cookie: similarity_session=<token>" \
4     https://similarityapp.ca/api/assignments/<assignmentId>
```

The `deleteAssignment` service method collects all object keys, deletes database records in a transaction, then best-effort deletes files from object storage.

Direct database purge (emergency use only, after API is unavailable):

Listing 7.2: Emergency direct purge

```
1 # Delete a specific assignment and cascade (Prisma manages FK
   # cascade)
2 psql $DATABASE_URL <<EOF
3 DELETE FROM "Assignment" WHERE id = '<assignmentId>';
4 EOF
5 # Note: this does NOT clean up object storage files.
6 # Manually identify and delete orphaned files from
   # OBJECT_STORAGE_ROOT.
```

7.3 Data Export Procedures

To export submission data for external analysis:

Listing 7.3: Export comparison results as CSV

```
1 psql $DATABASE_URL -c "COPY (  
2   SELECT  
3     a.title AS assignment,  
4     pr.\"similarityScore\",  
5     ls.\"displayName\" AS left_submission,  
6     rs.\"displayName\" AS right_submission,  
7     cr.\"completedAt\"  
8   FROM \"PairResult\" pr  
9   JOIN \"ComparisonRun\" cr ON pr.\"comparisonRunId\" = cr.id  
10  JOIN \"Assignment\" a ON cr.\"assignmentId\" = a.id  
11  JOIN \"Submission\" ls ON pr.\"leftSubmissionId\" = ls.id  
12  JOIN \"Submission\" rs ON pr.\"rightSubmissionId\" = rs.id  
13  ORDER BY pr.\"similarityScore\" DESC  
14 ) TO '/tmp/pair_results.csv' CSV HEADER;"  
15  
16 docker compose cp postgres:/tmp/pair_results.csv ./pair_results.  
   csv
```


Chapter 8. Security Maintenance

8.1 Access Control Review

User accounts are managed via the API. There is no dedicated admin console. To review all accounts:

Listing 8.1: Review user accounts

```
1 psql $DATABASE_URL -c \  
2 "SELECT id, email, role, \"firstName\", \"lastName\", \  
   createdAt\  
3 FROM \"User\" ORDER BY \"createdAt\" DESC;"
```

To deactivate a professor account (no soft-delete exists – this permanently deletes the account and all owned assignments):

Listing 8.2: Delete a professor account

```
1 # Via the API (professor self-deletes -- requires their session  
   token)  
2 # OR directly via database (emergency use only):  
3 psql $DATABASE_URL -c \  
4 "DELETE FROM \"User\" WHERE email = 'user@example.com' AND role  
   = 'PROFESSOR';"  
5 # This cascades to Session rows. Assignments are NOT  
   automatically deleted  
6 # unless FK cascade is set. Review schema for cascade rules.
```

NOTE: Student accounts are created implicitly when students upload with a logged-in session. There is no student account management UI.

8.2 Credential Rotation Schedule

Credential	Recommended rotation	How to rotate
PostgreSQL password	Annually / on compromise	Update <code>DATABASE_URL</code> in <code>.env</code> , redeploy
Redis password	Annually / on compromise	Add <code>requirepass</code> to Redis config, update
RSA identity key pair	On compromise / key expiry	See Section 3.5
Public upload token secret	Annually / on compromise	See Section 3.5
Bootstrap admin password	After initial setup	Via UI change-password flow
TLS certificates	Automatic (Caddy)	No action needed

8.3 Audit Log Review

NOTE: No dedicated audit logging is implemented. All auditable events are currently in application logs (Docker stdout/stderr).

Key events to search for in logs:

- POST /auth/login — Authentication attempts
- POST /auth/professor-signup — New account creation
- DELETE /auth/account — Account deletion
- POST /uploads/* — File uploads
- POST /comparison-runs — Comparison run creation
- GET /uploads/assignment/./identity — Identity reveal events

To extract identity reveal events from Docker logs:

Listing 8.3: Find identity reveal events in logs

```
1 docker compose logs api 2>/dev/null | grep "/identity"
```

Recommendation: Implement structured audit logging for identity reveal events, account deletions, and comparison run initiations. These events have regulatory/academic integrity implications.

8.4 Security Hardening Checklist

AUTH_COOKIE_SECURE=true is set in production

AUTH_COOKIE_DOMAIN is set to your specific domain

CORS_ALLOWED_ORIGINS lists only legitimate origins

PostgreSQL is not accessible from the public internet

Redis is not accessible from the public internet (no auth by default – add `requirepass`)

The RSA private key is stored as an environment variable, not in the Docker image

ADMIN_BOOTSTRAP_PASSWORD is removed from `.env` after bootstrap

NEXT_PUBLIC_API_BASE_URL uses HTTPS

Object storage root is on a volume with appropriate file-system permissions

Session expiry GC is scheduled (Section 3.7)

Rate limiting is implemented at Caddy or application layer